

Prize Bond Checking App — Package & Architecture Reference Guide

Every dependency in `pubspec.yaml`, what it does, why it was chosen, and where it is used in the app.

1. UI & Utility Packages

cupertino_icons (^1.0.8)

- **Purpose:** iOS-style icon set.
- **Why chosen:** ships by default with Flutter projects; keeps icons consistent if any Cupertino widgets are used.
- **Used in:** available app-wide (most icons are Material, this is a standard companion).
- **Problem solved:** icon coverage on iOS-style UI without custom assets.

intl (^0.20.2)

- **Purpose:** internationalization and date/number formatting.
- **Why chosen:** the standard Dart formatting library.
- **Used in:** `AppFormatDate` config class and all date displays — draw dates (`dd MMM, yyyy`), PDF export timestamps (`dd MMM yyyy, hh:mm a`), profile "Member Since" (`MMMM yyyy`).
- **Flow:** `DateTime` → `DateFormat('dd MMM').format(date)` → readable text.
- **Problem solved:** consistent, human-readable dates everywhere.

flutter_screenutil (^5.9.3)

- **Purpose:** responsive sizing — adapts the UI to every screen size.
- **Why chosen:** industry-standard responsive package; design once, run on any phone.
- **How it works internally:** you declare a design size (mine: **400 × 812**); at runtime it computes scale factors from the real screen and extensions like `.w`, `.h`, `.sp`, `.r` multiply your values by those factors.
- **Used in:** `ScreenUtilInit` wraps the whole app in `PrizeBondApp`; sizes throughout screens.
- **Problem solved:** no hardcoded pixels breaking on small/large phones.

flutter_svg (^2.0.10+1)

- **Purpose:** render SVG vector images.
 - **Why chosen:** SVGs stay sharp at any resolution and are tiny compared to PNGs.
 - **Used in:** confirmation page success graphic and other vector assets in `assets/images/`.
 - **Problem solved:** crisp graphics on all screen densities with one file.
-

2. State Management & Local Storage

get (^4.7.2) — GetX 🌟 (the backbone)

- **Purpose:** state management + dependency injection + navigation, in one package.
- **Why chosen:** minimal boilerplate for a solo project; reactive rebuilds; noBuildContext needed for navigation, snackbars, dialogs, bottom sheets.
- **How it works internally:** `.obs` wraps a value in an Rx stream; `Obx` widgets subscribe to every Rx they read and rebuild when it emits. `Get.put` stores instances in a global service locator; `GetMaterialApp` installs a navigator key so `Get.to` / `Get.back` work anywhere.
- **Used in:** every screen and all 14 controllers (`DrawController` , `AuthController` , `MarketplaceController` , `NavController` ...), all navigation, all snackbars/dialogs/bottom sheets.
- **Problem solved:** keeping UI in sync with data across the whole app with very little code.

get_storage (^2.1.1)

- **Purpose:** ultra-simple synchronous key-value storage (like SharedPreferences, but faster and by the GetX author).
 - **How it works internally:** keeps a memory map backed by a JSON file on disk; reads are instant, writes flush asynchronously.
 - **Used in:** settings toggles (`notifications_enabled` , `auto_check_enabled`), saved bonds list (`saved_bonds`), login session timestamp (`loginTime`), cached user profile (`user_profile_cache`), last logged-in UID, and the **marketplace offline pending queue** (`pending_marketplace_listings`).
 - **Problem solved:** instant local persistence for small data without async ceremony.
-

3. Firebase Packages

firebase_core (^3.6.0)

- **Purpose:** initializes the Firebase app connection; required by every other Firebase plugin.
- **Used in:** `main.dart` — `await Firebase.initializeApp()` is Step 1 of startup.

firebase_auth (^5.3.1)

- **Purpose:** user authentication.
- **Used in:** `AuthController.signInWithEmail()` , `SignupController.createUserAccount()` , sign-out, splash session check, guest detection (`currentUser == null`) on the My Bonds / Marketplace auth walls.
- **Flow:** email+password → Firebase verifies → returns UID → UID keys all user data.
- **Problem solved:** secure credential handling without writing any password/hashing code.

cloud_firestore (^5.4.4)

- **Purpose:** the main cloud database.
- **Used in:** `DrawService` (draws), `SavedBondService` (saved_bonds), `MarketplaceController` (marketplace), `AuthController` / `UserController` / `CustomerController` (customers), `NotificationService` (notifications).
- **Which screens depend on it:** Home, Draws, Draw Detail, My Bonds sync, Marketplace, Profile, Admin Dashboard, Create Draw, Manage Users.
- **Flow:** controller → service → `FirebaseFirestore.instance.collection(...).get()/add()/update()` → model `fromFirestore()` → Rx list → UI.

firebase_storage (^12.3.7)

- **Purpose:** cloud file storage for the admin's official draw PDFs.
- **Used in:** admin upload in `CreateDrawScreen` flow (path `draw_results/{denomination}/{drawNumber}.pdf`) and user downloads in `DrawController._downloadFromStorage` with live progress.
- **How it works internally:** upload/download tasks emit snapshot events with `bytesTransferred/totalBytes` — that's my progress bar.

firebase_messaging (^15.1.3)

- **Purpose:** push notifications (FCM).
 - **Used in:** `NotificationService` — token fetch + save to `customers/{uid}`, foreground listener (`onMessage`), background handler (`onBackgroundMessage`, top-level `@pragma('vm:entry-point')`), tap handling for background/terminated states, permission request.
 - **Flow:** Cloud Function sends → FCM routes to device token → handler shows a local notification → tap navigates to Draws or My Bonds by payload type.
 - **Problem solved:** users learn about new draws and personal wins even with the app closed.
-

4. Local Database (Offline Support)

hive (^2.2.3) + hive_flutter (^1.1.0)

- **Purpose:** fast local NoSQL database in "boxes".
 - **Why chosen:** pure Dart (no native code), very fast binary storage, perfect for caching document data. `hive_flutter` adds `Hive.initFlutter()` for easy path setup.
 - **How it works internally:** each box is an append-only binary file with an in-memory index; reads are memory-speed.
 - **Used in:** `OfflineCacheService` — boxes `draws_cache` (all draws, JSON-encoded by id), `saved_bonds_cache`, `app_settings` (last sync time, saved PDF paths).
 - **Which features depend on it:** offline bond checking, offline draws list, PDF path memory, "showing cached data" fallback when Firestore fails.
 - **Problem solved:** the app's core feature works with zero internet.
-

5. PDF & File Handling

file_picker (^8.1.2)

- **Purpose:** lets the admin pick a PDF file from the device.
- **Used in:** `AdminDrawController.pickPdf()` on the Create Draw screen (filtered to `.pdf`, shows name + size).
- **Flow:** admin taps attach → system file browser → `File` object → uploaded to Firebase Storage on publish.

path_provider (^2.1.4)

- **Purpose:** gives correct platform directories (temp, documents).
- **Used in:** `PdfExportService` — results exports go to the **temporary directory**; the schedule PDF goes to the **documents directory** so it persists offline; downloads land in app storage.

- **Problem solved:** file paths differ per OS; this abstracts them safely.

open_file (^4.5.0)

- **Purpose:** opens a file with the device's default app (PDF viewer).
- **Used in:** after every PDF download/generation — draw PDFs, results export, schedule PDF.
- **Why chosen:** actively maintained fork of open_file; handles Android/iOS intent differences.

pdf (^3.11.1)

- **Purpose:** generate PDF documents in pure Dart.
- **Used in:** PdfExportService (results report with a results table + per-draw winning numbers appendix; schedule PDF with next-draw dates) and DrawController._buildDrawPdf (single-draw PDF with a 6-column winning-number grid).
- **How it works internally:** a widget-like API (pw.MultiPage , pw.Table) laid out onto A4 pages, then serialized to PDF bytes.
- **Problem solved:** every draw has a shareable PDF even if the admin never uploaded one — and it works offline.

printing (^5.13.1)

- **Purpose:** print/share/preview PDF bytes via the OS print dialog and share sheet.
- **Used in:** companion to the pdf package for sharing generated documents.

6. Connectivity, Notifications, Permissions

connectivity_plus (^6.1.1)

- **Purpose:** detects network state (wifi / mobile / ethernet / none) and emits changes.
- **Used in:** ConnectivityService (GetXService, registered permanently) exposing reactive isOnline ; checked before Firestore loads, PDF downloads, bond sync, marketplace queue flush.
- **Flow:** OS network change → stream event → isOnline Rx updates → controllers pick Firestore vs Hive automatically.
- **Problem solved:** the decision-maker of the entire offline-first design.

flutter_local_notifications (^18.0.1)

- **Purpose:** shows notifications from the app itself on the device.
- **Used in:** NotificationService — creates Android channel prize_bond_channel ("Prize Bond Notifications", max importance), displays FCM messages in foreground/background, shows instant win alerts during manual checks, and handles notification taps (payload draw_result → Draws, winner → My Bonds).
- **Why needed alongside FCM:** FCM delivers the message; this package renders it with full control (channel, sound, priority) and handles foreground display, which FCM alone doesn't do on Android.

logger (^2.6.1)

- **Purpose:** pretty, leveled debug logging.
- **Used in:** controllers and services (DrawController , AdminDrawController , cloud sync paths) for development diagnostics.
- **Problem solved:** readable structured logs instead of bare print() .

permission_handler (^11.3.1)

- **Purpose:** requests and checks runtime permissions (camera, notifications).

- **Used in:** supporting the scanner's camera permission and notification permission flows.

mobile_scanner (^5.2.3) ★

- **Purpose:** QR/barcode scanning with the camera.
- **Why chosen:** simpler and far more reliable than a camera + ML Kit OCR pipeline (this reasoning is even commented in pubspec.yaml); actively maintained; good performance.
- **How it works internally:** streams camera frames into the platform barcode engine (Google ML Kit barcode API on Android, AVFoundation metadata output on iOS) and emits `BarcodeCapture` events with decoded values.
- **Used in:** `ScannerScreen` + `ScannerController` — `MobileScannerController(detectionSpeed: noDuplicates, facing: back)`; result validated against `^\d{6}$`, camera stopped on success, controller disposed in `onClose()`.
- **Flow:** Camera → frame analysis → barcode value → 6-digit validation → result view → "Use This Number" → Home quick-check field.

url_launcher (^6.3.1)

- **Purpose:** opens external apps/URLs (WhatsApp, dialer).
 - **Used in:** `MarketplaceController.contactSeller` — builds `https://wa.me/<+92 phone>?text=...` for WhatsApp chat, falls back to `tel:` for a phone call.
 - **Problem solved:** buyer-seller contact without building an in-app chat system.
-

7. Dev Dependencies

flutter_test — Flutter's testing framework (widget/unit tests).

flutter_lints (^5.0.0) — official lint rules that enforce Dart style and catch common mistakes at analysis time (configured via `analysis_options.yaml`).

8. Fonts & Assets

- **Poppins** font family bundled in 8 weights (declared in pubspec) — used by the auth screens' style layer; the main theme uses Material's Roboto with Material 3.
 - `assets/images/` — logo and SVG graphics.
-

9. Architecture Summary (quick reference)

Pattern

MVC + Service layer, implemented with GetX.

The three Cloud Functions

`onNewDraw` (broadcast + winner matching) · `onUserSignup` (hook) · `getAdminStats` (secure admin-only callable)